Opportunity App

A centralized hub connecting New Jersey students with volunteer and internship opportunities.

Description

The Opportunity App is a web platform built to bridge the gap between students seeking volunteer and internship experiences and the organizations that offer them. High school students across New Jersey can be required to complete volunteer hours to graduate, but finding meaningful opportunities is fragmented across dozens of websites, email chains, and bulletin boards — this app centralizes discovery, application, and communication in one place.

Students can browse and filter opportunities by type, location, skills required, and duration. They can follow organizations, apply directly through the platform, track their application status, send follow-up reminders to organizations, and log completed volunteer experiences on a personal profile. Organizations can post opportunities, manage applicants, send messages, and view impact metrics on their profile page.

The platform supports three user roles — student, organization, and administrator — each with a tailored dashboard and permissions. Administrators can review reported content and manage the opportunity approval queue. All three roles authenticate via email/password or Google Sign-In through the OAuth 2.0 flow.

The project was developed over a full semester as the CSCI 340 capstone at Drew University, built iteratively using Scrum with biweekly sprints, a GitHub Project board, and a pull-request-based review process.

Team Members

All team members contributed as full-stack developers, taking on backend, frontend, QA, and scrum responsibilities throughout the project.

Name	GitHub	Primary Role
Ryan DeVita	@modestmag-dev	Full-stack (Frontend focus) — Primarily responsible for UI design across the login workflow and the role-based dashboard.
Nguyen Tin Tin Do	@ndo1	Full-stack (Backend focus) — Primarily responsible for the AI-assisted messaging system and FAQ features.
Dev Nitinkumar Hirpara	@devhirpara29	Full-stack — Primarily responsible for the student and organization home dashboards and the follow organizations feature.
Brandon Jachera	@bjach04	Full-stack — Primarily responsible for opportunity search and filtering, home page UI redesign, and Google OAuth integration.

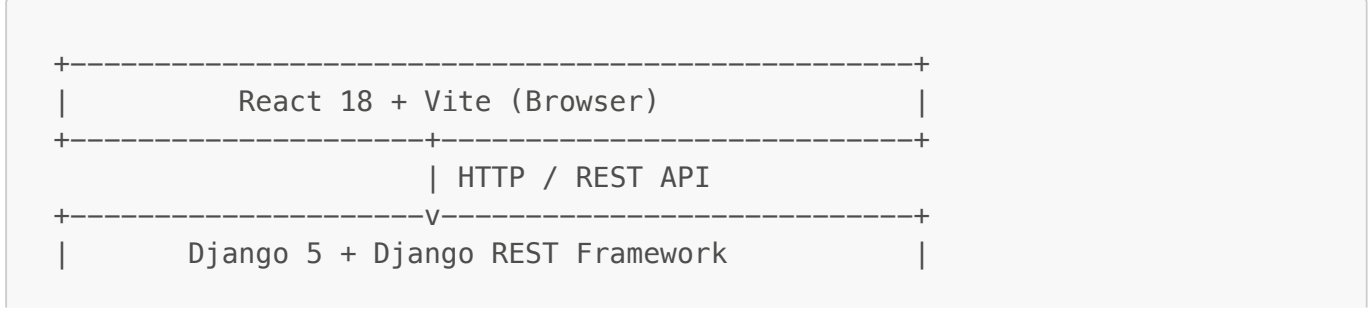
Name	GitHub	Primary Role
Maximillian Juliano	@mjuliano328	Full-stack (Backend focus) — Primarily responsible for volunteer messaging, opportunity management, and report content features.
Nakiwe McDonald	@nakiwem	Full-stack — Primarily responsible for user profiles, application tracking, and the Contact Us feature.
Puzi Wei	@pwei454	Full-stack (Backend focus) — Primarily responsible for the organization dashboard, application notifications, and remind organization feature.

Tech Stack

Layer	Technology
Backend Language	Python 3.12
Backend Framework	Django 5.2.12 + Django REST Framework
Frontend	React 18.3.1 + Vite 5.4.10 + Plain CSS
Database	PostgreSQL (Google Cloud SQL)
Auth	django-allauth 65.7.0 (email + Google OAuth 2.0)
DB Driver	psycopg2-binary 2.9.11
Config	python-dotenv 1.2.2
Frontend Testing	Vitest 3.2.4 + React Testing Library
Backend Testing	Django TestCase + coverage.py
Test DB	SQLite (auto-selected during `manage.py test`)
Deployment	Google Cloud (Cloud SQL)

Architecture Overview

The app uses Django's MVT architecture on the backend with a React + Vite frontend. The `accounts` app handles authentication, registration, and password reset. The `pages` app holds all core business logic — models, views, forms, and templates, including role-specific dashboards for students and organizations, and a Contact Us page for user support.



```
|                                                  |
|  +--------------+   +-------------------------+  |
|  |  accounts/   |   |        pages/           |  |
|  |  - User      |   |  Models:                |  |
|  |  - Register  |   |  - Opportunity          |  |
|  |  - Login     |   |  - Application          |  |
|  |  - Password  |   |  - Message/Notification |  |
|  |    Reset     |   |  - OrganizationProfile  |  |
|  |  - Google    |   |  - VolunteerProfile     |  |
|  |    OAuth     |   |  - ContactMessage       |  |
|  +--------------+   |  Views/Features:        |  |
|                     |  - Student Dashboard    |  |
|                     |  - Org Dashboard        |  |
|                     |  - Contact Us           |  |
|                     |  - Report Queue         |  |
|                     +-----------+-------------+  |
+-----------------------------------+--------------+-+
                                    | Django ORM
+-----------------------------------v--------------+
|          PostgreSQL (Google Cloud SQL)           |
+--------------------------------------------------+
```

Request flow: Browser → Django URL router → view function → ORM query → HTML template or JSON response.

Three user roles:

- **Student** — browse, apply, message organizations, track applications, send reminders, log achievements
- **Organization** — post opportunities, manage applicants, message students, view impact metrics
- **Administrator** — approve opportunities, manage reported content

---

Getting Started

Prerequisites

- Python 3.12+
- pip
- Node.js 18+ and npm (for the React frontend)
- Access to the shared PostgreSQL instance (via `.env`) — contact a team member for credentials

Installation

```
# 1. Clone the repository
git clone https://github.com/drew-csci/opportunity_app_srping_2026.git
cd opportunity_app_srping_2026

# 2. Create and activate a virtual environment
python3 -m venv venv
```

```
source venv/bin/activate        # macOS / Linux
# venv\Scripts\activate         # Windows

# 3. Install backend dependencies
pip install -r requirements.txt

# 4. Install frontend dependencies
cd frontend && npm install && cd ..
```

Configuration

Create a `.env` file in the project root (same directory as `manage.py`):

```
DJANGO_SECRET_KEY=dev-insecure-change-me
DJANGO_DEBUG=True
DB_NAME=opportunity_db
DB_USER=oppo_app
DB_PASSWORD=your_db_password
DB_HOST=34.16.174.60
DB_PORT=5432
ALLOWED_HOSTS=127.0.0.1,localhost
GOOGLE_CLIENT_ID=your_google_client_id
GOOGLE_CLIENT_SECRET=your_google_client_secret
```

> Never commit `.env` to version control. Contact a team member for the actual credential values.

Run the App

```
# Backend
python manage.py migrate
python manage.py runserver

# Frontend (in a separate terminal)
cd frontend && npm run dev
```

Visit http://127.0.0.1:8000

Test Accounts

Role	Email	Password
Student	student_oppo@drew.edu	1Opportunity!
Organization	org_oppo@drew.edu	1Opportunity!
Administrator	admin_oppo@drew.edu	1Opportunity!
Superuser	super_oppo@drew.edu	1OpportunityApp!

Admin panel: http://127.0.0.1:8000/admin

Usage

Start the app locally and navigate to the opportunity board as a student:

```
python manage.py runserver
# Visit http://127.0.0.1:8000/screen1/ to browse opportunities
```

Log in with the student test account (`student_oppo@drew.edu` / `1Opportunity!`), browse and filter opportunities by keyword, location, or type, apply to a listing, and track your submission at `/my-applications/`. Organizations log in separately and manage their applicant pipeline from the dashboard at `/org/dashboard/`.

Testing

Backend Tests

Tests use Django's built-in test runner. The test database automatically switches to SQLite when `test` is in `sys.argv` — no extra configuration needed.

```
# Run the full test suite
python manage.py test

# Run only the pages app tests
python manage.py test pages

# Run with full output
python manage.py test --verbosity=2

# Run with coverage report
coverage run manage.py test
coverage report
```

The suite in `pages/tests.py` covers:

- **Unit tests** — individual model methods and form validation
- **Integration tests** — view-to-database flows for application submission, messaging, and profile editing
- **Regression tests** — guard rails around previously fixed bugs (e.g., reminder status check)
- **Smoke tests** — quick pass/fail checks that key pages load without errors
- **Negative/edge-case tests** — invalid inputs, unauthorized access, boundary conditions

Test framework: Django's `unittest`-based `TestCase` and `Client`, with coverage.py for coverage reporting.

Frontend Tests

```
npx vitest
```

Project Management

GitHub Project Board: Opportunity App Spring 2026 Project

Branching Strategy

- `main` is the stable branch; all feature work happens on dedicated branches
- Naming: `feature/issue-<number>-short-description` or `<author>_<feature>`
- Direct pushes to `main` are not permitted — all changes go through pull requests

PR Review Process

- Each PR requires at least one reviewer approval before merging
- PRs are linked to GitHub Issues and user stories on the project board
- Merge conflicts are resolved on the feature branch before requesting review

---

Contributors

Contributor	GitHub Profile
Ryan DeVita	github.com/modestmag-dev
Nguyen Tin Tin Do	github.com/ndo1
Dev Nitinkumar Hirpara	github.com/devhirpara29
Brandon Jachera	github.com/bjach04
Maximillian Juliano	github.com/mjuliano328
Nakiwe McDonald	github.com/nakiwem
Puzi Wei	github.com/pwei454

Acknowledgments

Third-Party Libraries and Tools

- Django — web framework
- Django REST Framework — REST API layer
- React — frontend UI library
- Vite — frontend build tool
- django-allauth — authentication including Google OAuth 2.0
- psycopg2 — PostgreSQL adapter for Python

- python-dotenv — environment variable management
- Google Cloud SQL — hosted PostgreSQL database
- Vitest — frontend unit testing framework
- coverage.py — backend test coverage reporting

AI Tools

- **Claude Code (Anthropic)** — used by team members for debugging merge conflicts, writing test scaffolding, and reviewing code during development. All AI-generated suggestions were reviewed, tested, and integrated by the responsible team member.
- **GitHub Copilot** — used by some team members for code completion and boilerplate generation. All output was reviewed before inclusion.

AI tools assisted with development tasks. The design decisions, feature implementations, and work reflected in each team member's PRs represent their own meaningful contributions.